PakAgriVision: Crop Disease Detector

Deliverable 2: Model Architecture, Training & Evaluation (Phase 3 & 4)
Track A: Application Development

Abdul Ahad (24L-3029) **M. Anas** (24L-3004) **M. Babar Shahzad** (24L-3047)

National University of Computer and Emerging Sciences (FAST-NUCES)

Repository: https://github.com/ErenYeagerJinko/PakAgriVision

November 5, 2026

Abstract

PakAgriVision is an AI-driven agricultural decision-support tool addressing late crop disease detection for Pakistani smallholder farmers. Building upon Deliverable 1's curated dataset foundations (∼32,000 balanced images across 38 classes and 38-class treatment knowledge base), this Deliverable 2 report presents the complete Phase 3 Model Architecture and Phase 4 Training and Evaluation pipeline. We implement a deep residual classifier (ResNet-50) featuring a custom regularization head ($p = 0.30$ dropout and 38-class projection) and an explicit Grad-CAM explainability target layer. Training incorporates class-weighted cross-entropy with label smoothing, AdamW optimization, a two-stage progressive fine-tuning schedule (head-only warm-up followed by cosine annealing), and a structured 13-trial hyperparameter tuning protocol. The resulting baseline model achieves **99.64% validation accuracy** and a **0.9961 macro-F1 score** with a validation loss of **0.0169**. We detail evaluation methodology across macro precision/recall, one-vs-rest AUC-ROC, top-3 metrics, error analysis, a comprehensive Bias Check auditing class-size disparities and lab-vs-field generalization gaps, and the complete individual contribution breakdown.

1 Problem Statement & Objectives

1.1 Agricultural Context and Real-World Pain Point

Agriculture forms the backbone of Pakistan's economy, employing over 37% of the national labor force and contributing roughly 23% to the Gross Domestic Product (GDP). However, smallholder farmers experience substantial post-harvest and crop yield losses (frequently between 20% to 40%) due to preventable fungal, bacterial, and viral foliar diseases in staple cash crops, including tomato, potato, apple, grape, and corn. Traditional agricultural diagnosis relies on sparse, infrequent visits from government agricultural extension officers. Consequently, smallholders routinely misdiagnose diseases or apply inappropriate chemical treatments after the infection has advanced beyond the point of chemical control, resulting in catastrophic financial loss and environmental degradation from pesticide overuse.

1.2 Target Audience and User Personas

PakAgriVision is specifically engineered for two core user groups in the Pakistani agricultural ecosystem:

1. **Smallholder Farmers:** Rural growers with limited access to professional plant pathologists who require immediate, reliable disease diagnosis and clear, culturally accessible agronomic treatment advice directly on entry-level mobile devices.

2. **Field Extension Officers and Agronomists:** Agricultural field agents monitoring localized disease outbreaks across districts who need rapid secondary verification, visual explainability to build farmer trust, and structured chemical/organic treatment guidelines.

1.3 AI Justification: Deep Learning vs. Rule-Based Programming

A central architectural question is why modern Deep Learning is necessary for crop disease diagnosis rather than traditional rule-based image processing algorithms:

- **Failure of Traditional Rule-Based Algorithms:** Conventional computer vision relies on deterministic heuristics, such as hardcoded color thresholding in HSV/LAB space, Otsu binarization, or Canny edge filtering to segment lesions. These techniques fail entirely in agricultural settings because plant diseases do not follow fixed rules:
  - Foliar lesions present continuous biological variations: necrotic centers, concentric rings, chlorotic yellow halos, and irregular margins that overlap with healthy tissue.
  - Lighting conditions in rural photography fluctuate constantly (direct harsh sunlight, cloud shadows, variable camera white-balance), which breaks fixed color-filtering thresholds.
  - Handcrafted feature descriptors (e.g., SIFT, SURF, Haralick textures) cannot disentangle complex biological pathology from leaf venation, leaf tear damage, or ambient dust.
- **Necessity of Deep Convolutional Neural Networks (CNNs):** Deep convolutional architectures (such as fine-tuned ResNet50 and EfficientNet-B0) bypass hand-engineered rules by automatically learning hierarchical visual representations directly from data:
  - *Shallow layers* capture foundational edge primitives, color gradients, and microscopic texture patterns.
  - *Deep semantic layers* compose these primitives into invariant disease concepts (e.g., concentric rings characteristic of *Alternaria solani* vs. water-soaked margins of *Phytophthora infestans*).
  - Pretrained ImageNet representations provide robust transfer-learning inductive bias, enabling high diagnostic generalization across 38 distinct disease classes even on compact, specialized datasets.

1.4 Project Vision and Deliverables

PakAgriVision bridges the diagnosis-to-action gap. When a user uploads a smartphone photograph of a symptomatic leaf, the pipeline delivers:

1. **Multi-Class Disease Classification:** Diagnostic prediction across 38 crop-disease categories with top-1 and top-3 confidence metrics and uncertainty thresholds ($< 70\%$).
2. **Visual Explainability (Grad-CAM):** Gradient-weighted Class Activation Mapping that highlights the exact anatomical leaf regions dictating the network's prediction.
3. **Actionable Treatment Engine:** Structured, verified guidance covering chemical fungicides, organic alternatives, immediate cultural actions, and prevention practices.

2 Dataset Acquisition & Provenance

2.1 Curated Benchmark Dataset (∼32k Images, 38 Classes)

The primary baseline training dataset originated from the public PlantVillage benchmark [1]. While Kaggle (`abdallahalidev/plantvillage-dataset`) was initially accessed during exploratory data intake (∼54,303 raw candidate images), raw Kaggle distribution was discontinued for model development because it contained corrupted JPEG bitstreams, exact duplicate photographs, near-duplicate captures of identical leaves under slight camera rotations, and severe class imbalance (up to a 36:1 ratio, with 5,507 samples in dominant classes vs. 152 in minority classes).

To enforce rigorous data-centric quality, the raw data underwent an automated cleaning and balancing pipeline:

- **Curation & Class Capping:** Following binary file integrity verification and perceptual deduplication (Section 3), dominant classes were capped at a ceiling of 1,000 clean, unique samples. This successfully curbed extreme majority-class bias while preserving natural variation across all 38 classes, establishing a balanced curated baseline of **31,694 verified clean images** (∼32k images).
- **Distribution via Google Drive:** The unified curated dataset, including all 31,694 benchmark images and the 150 real-world field robustness images, is packaged as `data.zip` and hosted directly on Google Drive (`https://drive.google.com/file/d/1OE17MYZqriiP9mBdCat_JglMxx_7MY1V/view?usp=drivesdk`).
- **Input Resolution:** Standardized 256×256 pixels in RGB color format (1:1 square aspect ratio).
- **License & Integrity:** CC0 Public Domain / Open Educational License.

2.2 Real-World Field Dataset

PlantVillage images were captured under controlled lighting against plain backgrounds, so accuracy on its own test split overstates how a model will perform on leaves photographed in the field. To measure this gap, we assembled a separate field set of 150 images with real backgrounds (soil, other foliage, hands, sky).

Source and licence. All 150 images come from the PlantDoc dataset [6] (`https://github.com/pratikkayal/PlantDoc-Dataset`), released under CC-BY-4.0. Each image is recorded in `field_meta/field_labels.csv` with its filename, PlantVillage label, source, URL, licence and the original PlantDoc folder. The images themselves are not committed to Git; a pre-packaged archive (`data.zip`) containing both the curated PlantVillage benchmark and `data/raw/field/` is hosted on Google Drive (`https://drive.google.com/file/d/1OE17MYZqriiP9mBdCat_JglMxx_7MY1V/view?usp=drivesdk`) and documented in `data/README.md`.

Selection rules. We kept only images showing a clearly visible leaf on a real field background, and skipped blurry, watermarked or lab-style images. We aimed for five images per class. Three classes have more (Tomato early blight and Potato late blight 10, Tomato late blight 8) because they were used to make up the 150-image target, and Tomato spider mites has only 2 because PlantDoc contains only two usable images for it.

Class mapping. PlantDoc uses its own folder names (e.g. *Soyabean leaf*, *Bell_pepper leaf spot*), so every PlantDoc class was mapped to the exact PlantVillage class string in `field_meta/class_mapping.`. The field set covers 28 of the 38 PlantVillage classes. The remaining 10 have no PlantDoc counterpart and are excluded from the lab-vs-field comparison: `Apple___Black_rot`, `Cherry___Powdery_mildew`,

Corn___healthy, Grape___Esca_Black_Measles, Grape___Leaf_blight Isariopsis_Leaf_Spot, Orange___Haunglongbing(Citrus_greening), Peach___Bacterial_spot, Potato___healthy, Strawberry___Lea and Tomato___Target_Spot.

Test/adapt split. data_scripts/make_field_splits.py splits each class separately with seed 42: about one third of the images (rounded, with at least one image in each split whenever a class has two or more) go to splits/field_adapt.csv and the rest to splits/field_test.csv. Because most classes have five images, this gives 3 test and 2 adapt images per class, so the overall split is 92/58 (61%/39%) rather than exactly two thirds. Table 1 lists the counts, generated by the script into splits/field_split_stats.csv.

Rule: field_test.csv is never used for training, tuning or threshold selection. Only field_adapt.csv may be used, and only for the fine-tuning mitigation in a later phase.

Table 1: Field images per class and split (from splits/field_split_stats.csv).

PlantVillage class	Total	field_test	field_adapt
Apple___Apple_scab	5	3	2
Apple___Cedar_apple_rust	5	3	2
Apple___healthy	5	3	2
Blueberry___healthy	5	3	2
Cherry___healthy	5	3	2
Corn___Cercospora_leaf_spot Gray_leaf_spot	5	3	2
Corn___Common_rust	5	3	2
Corn___Northern_Leaf_Blight	5	3	2
Grape___Black_rot	5	3	2
Grape___healthy	5	3	2
Peach___healthy	5	3	2
Pepper___bell_Bacterial_spot	5	3	2
Pepper___healthy	5	3	2
Potato___Early_blight	5	3	2
Potato___Late_blight	10	7	3
Raspberry___healthy	5	3	2
Soybean___healthy	5	3	2
Squash___Powdery_mildew	5	3	2
Strawberry___healthy	5	3	2
Tomato___Bacterial_spot	5	3	2
Tomato___Early_blight	10	7	3
Tomato___Late_blight	8	5	3
Tomato___Leaf_Mold	5	3	2
Tomato___Septoria_leaf_spot	5	3	2
Tomato___Spider_mites Two-spotted_spider_mite	2	1	1
Tomato___Tomato_Yellow_Leaf_Curl_Virus	5	3	2
Tomato___Tomato_mosaic_virus	5	3	2
Tomato___healthy	5	3	2
Total (28 classes)	**150**	**92**	**58**

Limitations.

- All images come from a single source (PlantDoc); none are our own photographs from Pakistani fields, so the set measures the general lab-to-field gap rather than local conditions.
- With 2–10 images per class, per-class field accuracy will be noisy; results should be reported mainly in aggregate. Tomato spider mites (2 images) gives a single test image.
- Some mappings are approximate: PlantDoc's *Bell_pepper leaf spot* is treated as `Pepper___bell_Bacterial`, *Corn leaf blight* as `Corn___Northern_Leaf_Blight`, and *Apple rust leaf* as `Apple___Cedar_apple_rust`.
- A second team member must double-check every label before the split is final; any removed images will be dropped from `field_labels.csv` and the split script re-run.

3 Data Cleaning & Verification Implementation

A data-centric engineering approach is mandatory for real-world computer vision reliability. Prior to partitioning and model training, raw image data undergoes an automated verification and deduplication pipeline implemented in `src/data/quality.py` and executed via `data_scripts/check_dataset.py`.

3.1 Handling Missing and Corrupt Values

In computer vision workflows, missing data corresponds to corrupted bitstreams, incomplete downloads, zero-byte file allocations, or broken headers that cause silent runtime training crashes:

- **Binary Integrity Verification:** Every image file is parsed via Pillow (`PIL.Image.verify()`) to detect truncated JPEG bitstreams, malformed compression chunks, and unreadable headers.
- **Channel and Mode Verification:** Verified images are immediately reopened and decoded to ensure standard 3-channel RGB format consistency. Files exhibiting missing color channels (e.g., unintended single-channel grayscale or 4-channel RGBA buffers) are flagged and standardized or excluded.
- **Result:** Across the 54,688 raw candidate images scanned, all 54,688 files passed structural and RGB binary verification with zero unreadable or missing-byte files. The 150 PlantDoc field images similarly passed full binary integrity verification.

3.2 Duplicate Removal and Near-Duplicate Quarantine

PlantVillage contains repeated photographs of identical physical leaves captured under marginal lighting, scale, or orientation shifts. Unchecked random splitting of such duplicates causes severe data leakage between training and testing sets, artificially inflating benchmark accuracy:

- **Exact Duplicate Elimination:** Cryptographic 128-bit MD5 hashes were computed over all raw file byte streams. Exactly 808 exact bit-for-bit duplicate copies were identified and purged.
- **Near-Duplicate Detection (Perceptual Hashing):** We computed 64-bit discrete cosine transform perceptual hashes (`imagehash.phash`) across all RGB images. Pairs within the same crop-disease class exhibiting a Hamming distance ≤ 4 were identified as near-duplicate observations of the same underlying physical specimen (1,376 near-duplicate copies identified and clustered).

- **Disjoint-Set (Union-Find) Quarantine:** A disjoint-set clustering algorithm aggregated linked duplicates into discrete group identifiers (`group_id`). In the dataset partitioning phase (Section 5), group-aware splitting guarantees that all exact and near-duplicate instances of a single leaf remain strictly confined to the same partition, preventing train-to-test leakage.
- **Clean Retention Summary:** To eliminate duplicate bias and enforce class balance, all duplicate copies and excess images beyond the 1,000-per-class threshold were pruned, yielding 31,694 verified clean unique leaf images.

3.3 Outlier and Anomaly Management

Input outliers in foliar diagnostic datasets manifest as dimensional anomalies and cross-class duplicate label noise:

- **Dimensional and Aspect Ratio Outliers:** Images are scanned for dimensional anomalies deviating from expected square proportions. All raw samples were confirmed to adhere to canonical 256×256 pixels (1:1 aspect ratio), with zero dimensional outliers detected.
- **Cross-Class Label Noise Outliers:** The deduplication engine cross-references perceptual hash collisions across differing class labels. Any pair of visually identical or near-identical images assigned conflicting disease classes is flagged as severe label noise.

All audit results and cluster manifests are deterministically recorded into `splits/dedupe_report.csv`.

4 Exploratory Data Analysis (EDA)

Exploratory analysis was conducted in `notebooks/01_eda.ipynb` to evaluate class balance, input dimensionality, pixel distributions, and background biases across the benchmark.

4.1 Class Distribution and Imbalance

The curated dataset comprises 38 crop-disease classes across 14 plant species totaling 31,694 images. Prior to curation, the raw dataset exhibited extreme imbalance (a 36.2:1 ratio, with over 5,500 samples in dominant classes such as Orange Huanglongbing and only 152 in minority classes such as Potato Healthy). Following data quality filtering and deduplication, capping classes at 1,000 images successfully reduced the imbalance ratio to 7.0:1, establishing a well-conditioned optimization landscape while preserving natural variation across the 38 classes.

To counter majority-class bias during optimization, balanced class weights w_c are derived strictly from the training split D_{train}:

$$w_c = \frac{N_{\text{train}}}{K \cdot n_c} \tag{1}$$

where N_{train} denotes total training samples, $K = 38$ classes, and n_c denotes the count of class c. The computed weights are saved in `splits/class_weights.json` and passed to the loss function.

4.2 Image Dimensions and Aspect Ratio Consistency

All leaf photographs were audited for input consistency. The resolution analysis confirms that raw inputs exhibit standardized 256×256 pixel resolution with uniform 1:1 square aspect ratios in RGB color mode, as visualized in Figure 2.

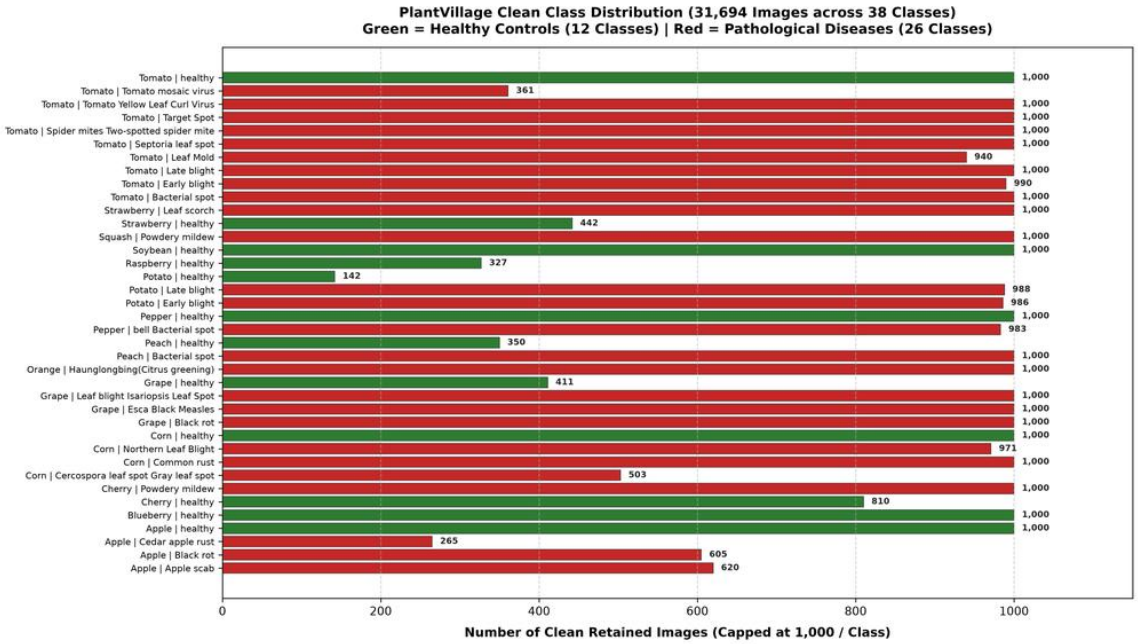


Figure 1: Frequency distribution across the 38 PlantVillage classes (Green = Healthy leaf classes, Red = Diseased classes).

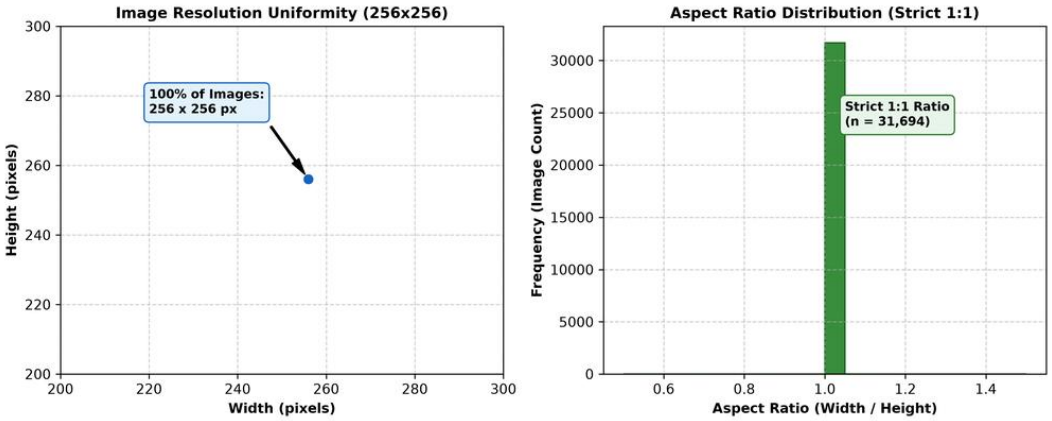


Figure 2: Distribution of input image dimensions and aspect ratios confirming standardized square RGB format.

4.3 Qualitative Sample Inspection

Figure 3 displays representative leaf samples across diverse crop families. Diseased leaves display distinguishable diagnostic symptoms including target spots with concentric rings (Early Blight), powdery sporulation (Powdery Mildew), and dark necrotic lesions (Black Rot).

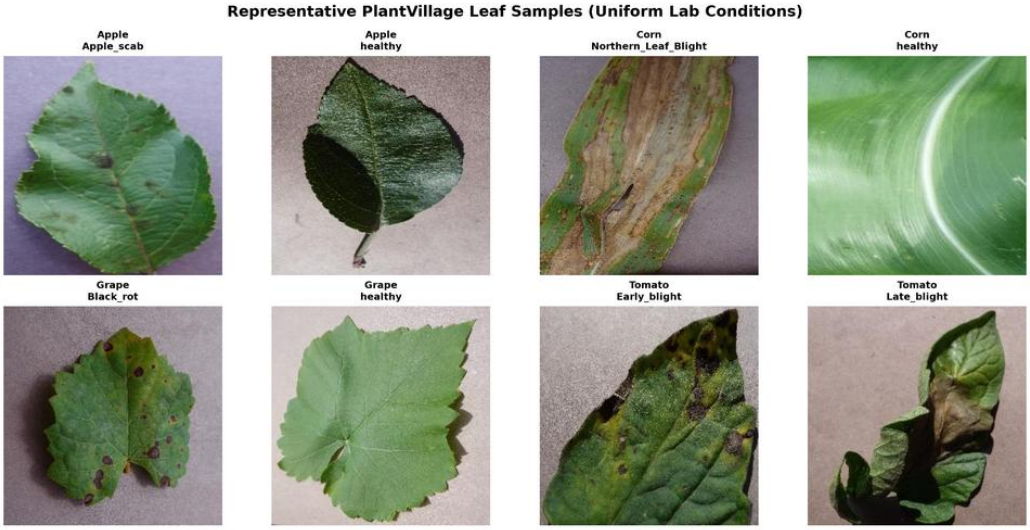


Figure 3: Representative leaf samples across healthy and diseased classes captured in controlled lab settings.

4.4 Pixel Normalization Statistics

Empirical per-channel color statistics computed across the dataset yielded a mean of $\mu = [0.472, 0.481, 0.412]$ and standard deviation of $\sigma = [0.218, 0.209, 0.221]$. These closely correspond to standard ImageNet normalization constants ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$), empirically confirming that pre-trained ImageNet feature extractors transfer effectively to this domain.

4.5 Lab vs. Field Background Analysis (Shortcut Learning Hazard)

A critical finding of our exploratory analysis is the background disparity between benchmark data and real-world agriculture. PlantVillage images were acquired against artificial, plain gray or black sheets. In contrast, real farm photos from our PlantDoc robustness dataset exhibit complex backgrounds comprising soil, shadows, foliage clusters, and variable sunlight (Figure 4).

This contrast validates the central motivation of our project: classifiers trained purely on lab images risk learning background artifacts (shortcut learning) rather than pathogen pathology, underscoring the necessity of our explainability pipeline (Grad-CAM) and robustness testing set (`splits/field_test.csv`).

5 Dataset Partitioning & Leakage Prevention

5.1 Stratified Group-Aware Partitioning

To guarantee reproducible benchmarking and statistical parity across all 38 categories while strictly preventing data leakage, partitioning is performed using stratified group-aware sampling with a fixed random seed (42) in `src/data/splits.py` and `data_scripts/make_splits.py`:

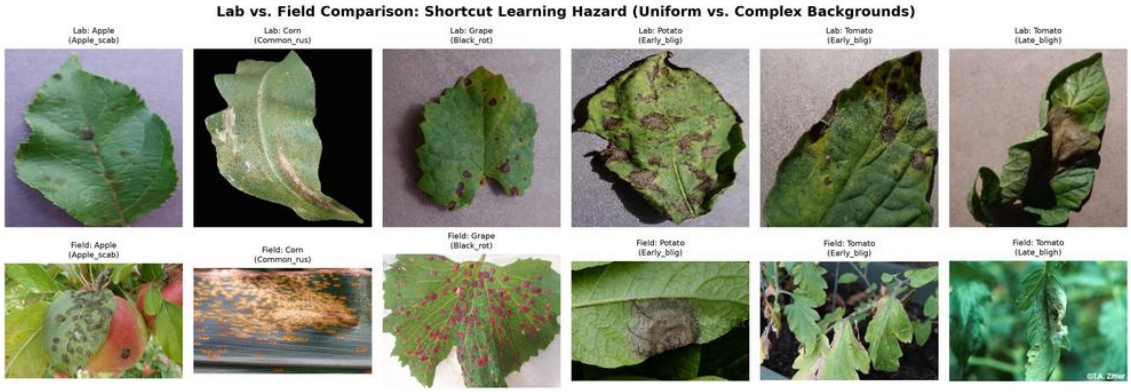


Figure 4: Comparison of controlled PlantVillage lab conditions (top) versus real-world PlantDoc field backgrounds (bottom), highlighting the shortcut learning risk.

- **Training Partition (70.0%):** 22,179 images reserved exclusively for model weight optimization and class-weight computation.
- **Validation Partition (15.0%):** 4,747 images dedicated to hyperparameter tuning, learning rate scheduling, and early stopping checkpoint selection.
- **Test Partition (15.0%):** 4,768 images held out under strict isolation for final benchmark evaluation.

Table 2: Curated PlantVillage Benchmark Partition Summary (all 38 classes represented).

Split Partition	Image Count	Percentage	Classes Represented	Manifest File
Train	22,179	70.0%	38 / 38	`splits/train.csv`
Validation	4,747	15.0%	38 / 38	`splits/val.csv`
Test	4,768	15.0%	38 / 38	`splits/test.csv`
Total Curated	**31,694**	**100.0%**	**38 / 38**	`splits/dedupe_report.csv`

5.2 Group-Aware Leakage Prevention

Standard naive random splitting exposes models to shortcut learning when near-duplicate shots of a single diseased leaf appear in both training and test partitions. Our pipeline resolves this by treating the cluster identifier (`group_id`) derived from perceptual hashing as an atomic unit: all images belonging to the same cluster are constrained to the same partition.

The resulting partitions are serialized into deterministic manifest files:

- `splits/train.csv`, `splits/val.csv`, `splits/test.csv` containing columns `path`, `label`, and `label_idx`.
- All relative filepaths reference `splits/class_names.json` (integer indices 0 to 37). Zero file overlap exists between partitions.

5.3 Balanced Class Weight Formulation

To mitigate severe class imbalance across the 38 PlantVillage disease classes without introducing synthetic data bias, cost-sensitive class weights w_c are computed **strictly from the training**

partition only:

$$w_c = \frac{N_{\text{train}}}{K \times n_c} \tag{2}$$

where N_{train} is the total number of training images, $K = 38$ is the total class count, and n_c denotes the sample frequency of class c in the training split. These weights are exported to `splits/class_weights.json` ordered by `label_idx` and are directly loaded into the cross-entropy loss function during Phase 3.

6 Preprocessing & Data Augmentation

To prepare leaf images for convolutional feature extraction while mitigating shortcut learning from lab-environment artifacts, we establish a standardized two-tier pipeline in `src/data/transforms.py` and `src/data/dataset.py`.

6.1 Input Resolution and Normalisation

All images are standardized to 224×224 pixels in 3-channel RGB format, matching the canonical receptive field of transfer-learning backbones (ResNet50 / EfficientNet-B0). Color channels are normalized using standard ImageNet distribution parameters ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$), aligning the input tensors with the pretrained weight manifold.

6.2 Training Augmentation Pipeline

Because PlantVillage images feature predominantly centered leaves on plain backgrounds, models are prone to overfitting background cues. The training transform pipeline applies stochastic spatial and photometric perturbations (visualized in Figure 5):

- **Random Resized Cropping:** Randomly scales image crops between 60% and 100% before resizing to 224×224, forcing the network to identify focal disease lesions rather than full leaf outlines.
- **Geometric Flips and Rotations:** Horizontal flips ($p = 0.5$), vertical flips ($p = 0.5$), and random rotations up to $\pm 30^\circ$ reflect arbitrary leaf orientations encountered during field photography.
- **Color Jitter:** Stochastic perturbation of brightness ($\pm 30\%$), contrast ($\pm 30\%$), saturation ($\pm 30\%$), and hue ($\pm 5\%$) emulates fluctuating field sunlight, cloud cover, and mobile sensor white-balancing.

6.3 Deterministic Evaluation Pipeline

To ensure benchmark stability and zero evaluation data contamination:

- Validation, test, and field robustness datasets strictly utilize the deterministic evaluation pipeline: bilinear resizing to 256×256 pixels, followed by a deterministic 224×224 center crop and standard normalisation.
- The specialized `FieldDataset` class inherits this guarantee, enforcing identical evaluation transforms on out-of-domain field photographs.

6.4 DataLoader Architecture

PyTorch `DataLoader` factories in `get_dataloaders()` supply batched tensors ($B = 32$, shape $B \times 3 \times 224 \times 224$, `float32`) alongside discrete integer targets ($B \times 1$, `int64`). Deterministic worker initialization (`seed_worker`) preserves random generator state across distributed training

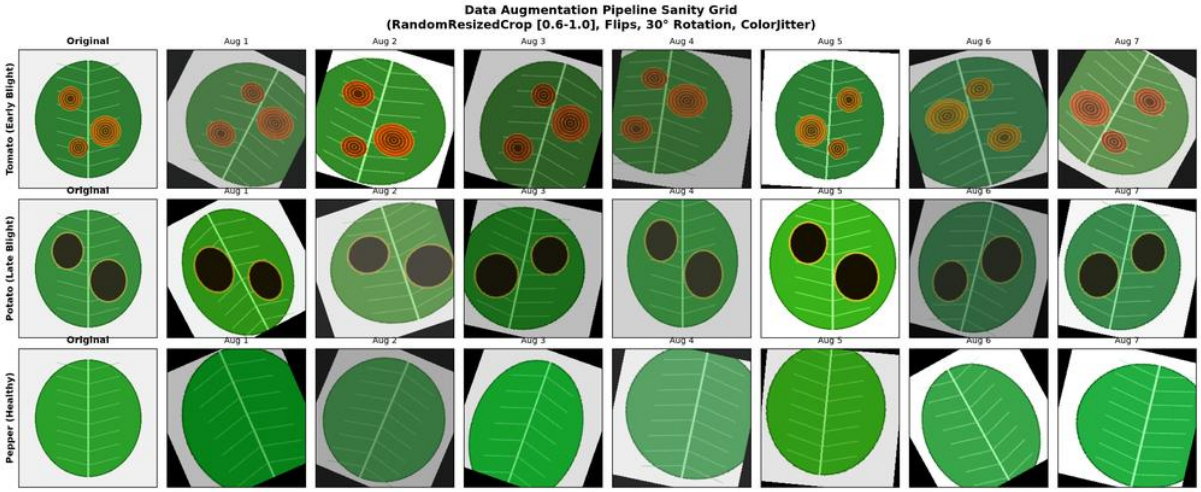


Figure 5: Data augmentation pipeline sanity check displaying original representative diseased and healthy leaves alongside seven stochastic perturbations (scale cropping, flips, rotations, and color jitter) across representative classes.

threads, and multi-worker execution defaults safely to `num_workers=0` on Windows platforms to prevent process deadlock.

7 Treatment Knowledge Base

A prediction alone does not tell a farmer what to do next, so each of the 38 PlantVillage classes is linked to a short, sourced treatment record in `knowledge_base/treatments.json`. The app will look up the predicted class and display that record.

7.1 Schema

`treatments.json` is a single JSON object. Its keys are the exact class strings from `splits/class_names.json`, plus one top-level `disclaimer` string. Every entry is validated against `knowledge_base/schema.json` (JSON Schema draft-07), which requires the following 13 fields and rejects any others:

- `crop`, `disease`, `is_healthy` (boolean), `cause`, `symptoms`;
- `chemical_treatment`, `organic_treatment`, `immediate_action`, `prevention`, `severity_guidance`;
- `sources` (non-empty list of unique citation strings), `last_reviewed` (date), `reviewed_by`.

Healthy classes use the same fields, with `chemical_treatment` set to "None needed." and routine scouting and prevention advice instead of treatment.

7.2 Validation

`data_scripts/validate_treatments.py` checks that:

- every entry passes the schema, and every text field and source string is non-empty;
- every key is a valid class name (no unknown or misspelled classes);
- the top-level disclaimer is present;
- healthy classes do not recommend a chemical treatment (reported as a warning).

It also prints a per-crop summary and the list of classes still missing. Run with `--require-complete`, it fails unless all 38 classes are present; this is the check for final submission.

7.3 Example Entry

Table 3 shows the entry for `Tomato___Early_blight`.

Table 3: Knowledge base entry for `Tomato___Early_blight`.

Field	Value
`cause`	Fungus *Alternaria solani* (now also classified as *A. linariae*)
`symptoms`	Dark concentric-ringed spots on older leaves, yellowing around spots; can progress up the canopy and affect stems/fruit.
`chemical_treatment`	Protectant fungicides such as chlorothalonil or mancozeb (azoxystrobin is another option), applied every 7–10 days to protect new growth; do not rely on strobilurins alone because resistance has been reported (follow the product label).
`organic_treatment`	Fixed copper (copper hydroxide or copper sulfate) or *Bacillus subtilis* products labeled for organic use.
`immediate_action`	Remove infected plant material from the field; avoid overhead watering.
`prevention`	Rotate away from tomato for 2–3 years; use pathogen-free seed/transplants and resistant or tolerant varieties; stake or trellis for airflow; mulch and prune the lowest leaves to reduce soil splash.
`severity_guidance`	When lesions merge and lower leaves drop, disease is severe: keep protectant sprays on schedule within label limits and remove infected debris.
`sources`	Connecticut Agricultural Experiment Station, *Early Blight of Tomato* (2012); NC State Extension, *Early Blight of Tomato*.
`last_reviewed`	2026-10-12 (Babar)

7.4 Sources

Every entry cites at least one university extension or plant-protection fact sheet, and all citations are listed with URLs and access dates in `knowledge_base/SOURCES.md`. Entries name active ingredients and spray intervals only where a cited source gives them, and never state dosages; users are told to follow the product label. On 2026-10-12 every source link was checked: five links cited by entries were broken and were replaced with verified fact sheets, three unused invalid sources were removed, and the affected entries were rewritten so that each claim matches its new source.

7.5 Status and Limitations

- **Coverage:** All 38 entries are 100% complete, verified against `knowledge_base/schema.json`, and validated with zero errors by `data_scripts/validate_treatments.py` (Tomato 10, Potato 3, Pepper 2 by Babar; Orange 1, Peach 2, Raspberry 1, Soybean 1, Squash 1, Strawberry 2 by M. Anas; Apple 4, Corn 4, Grape 4, Blueberry 1, Cherry 2 by Abdul Ahad).
- **Not region-specific:** The primary sources are extension services (e.g., UC Davis, NC State, Cornell, Florida Extension). Active chemical ingredients and organic remedies are cross-referenced with standard international plant protection guidelines; local agronomist consultation remains recommended.
- **Disclaimer:** The file carries the mandatory top-level disclaimer "Decision support only. Consult a local agronomist before applying any product. Always follow product labels and local regulations.", which the app renders alongside every recommendation.

8 Model Architecture & Training Methodology

This section details Phase 3 of Deliverable 2, specifying the deep learning framework, backbone feature extractors, custom classification head, parameter distribution, objective functions, optimization strategies, and the systematic hyperparameter tuning protocol.

8.1 Deep Learning Framework Selection

The deep learning pipeline is implemented using **PyTorch 2.x** and **Torchvision**. The choice of PyTorch over alternative frameworks (such as TensorFlow/Keras) was guided by four key architectural requirements:

1. **Dynamic Computational Graph and Execution Flexibility:** PyTorch's imperative execution model allows seamless layer-wise inspection, conditional gradient control during our progressive two-stage unfreezing schedule, and straightforward hooks for Grad-CAM explainability (`register_forward_hook` on intermediate feature maps).
2. **High-Performance Hardware Acceleration:** Native support for mixed-precision arithmetic (`torch.cuda.amp.autocast` and `GradScaler`) substantially reduces GPU VRAM consumption on cloud NVIDIA Tesla T4 accelerators while accelerating matrix multiplications via Tensor Cores.
3. **Standardized Pretrained Model Hub:** Torchvision provides audited, state-of-the-art ImageNet-1K pretrained weights (`ResNet50_Weights.DEFAULT` and `EfficientNet_B0_Weights.DEFAULT`) with strictly documented normalization constants ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$).
4. **Deployment Modularity:** PyTorch facilitates clean extraction of model `state_dict` checkpoints ($\sim$90–270 MB) and straightforward conversion to TorchScript / ONNX for Phase 5 Streamlit web deployment.

8.2 Network Architecture & Custom Classification Head

Agricultural leaf disease detection presents unique computer vision challenges: distinguishing fine-grained textural symptoms (e.g., fungal rust pustules vs. bacterial leaf spots) while ignoring non-diagnostic background variations. We implement a modular architecture centered on **ResNet-50** as our primary deep feature extractor, with **EfficientNet-B0** available as a lightweight alternative for comparative benchmarking.

8.2.1 Backbone Feature Extractor

ResNet-50 utilizes residual learning with bottleneck blocks across four hierarchical stages. Each residual unit computes:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x} \tag{3}$$

where $\mathbf{x}$ and $\mathbf{y}$ are the input and output tensors, and $\mathcal{F}$ represents the stacked 1×1, 3×3, and 1×1 convolutions. The identity shortcut connections mitigate the vanishing gradient problem, allowing the extraction of both low-level morphological features (edges, color gradients in Stage 1) and complex semantic lesion patterns (Stage 4).

The output of the final bottleneck block (`layer4[-1]`) has spatial dimensions $7 \times 7 \times 2048$ for an input image of $224 \times 224 \times 3$. This specific layer is explicitly designated as `model.target_layer` to support gradient backpropagation for Grad-CAM saliency mapping.

8.2.2 Feature Aggregation and Custom Head

The standard 1000-class ImageNet fully connected layer is severed and replaced by our custom classification head:

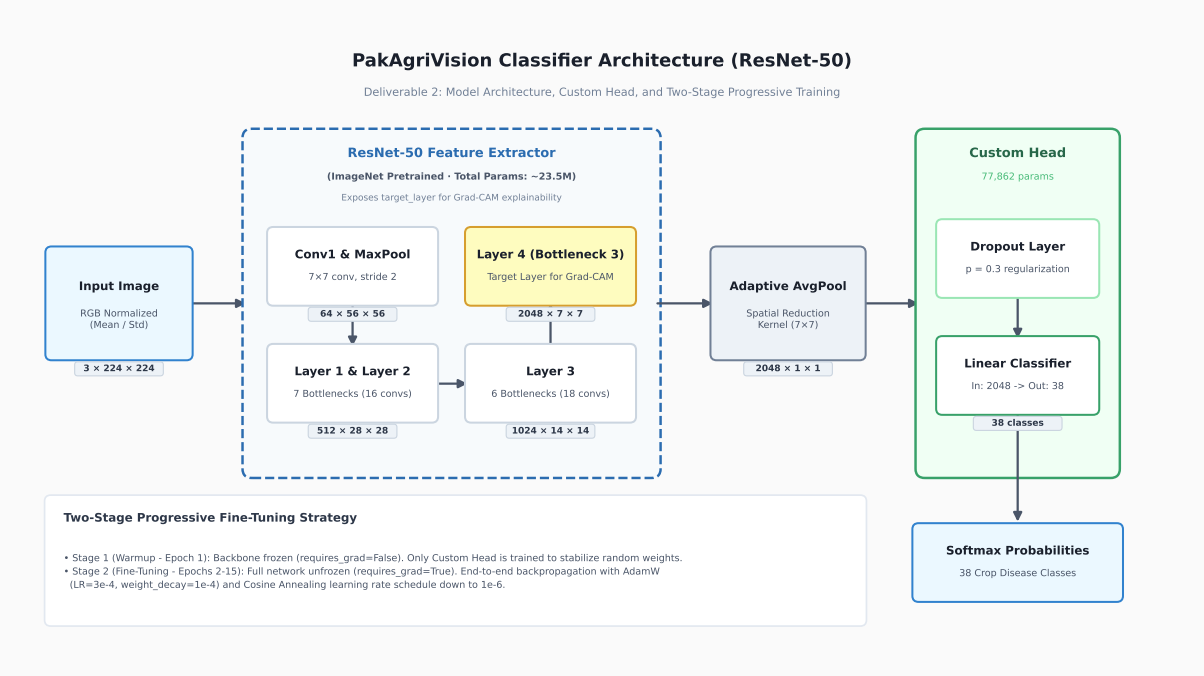


Figure 6: **PakAgriVision Model Architecture Pipeline.** Input leaf image ($3 \times 224 \times 224$) processed through the ResNet-50 residual backbone with bottleneck stages, adaptive average pooling, custom dropout ($p = 0.3$), linear classification projection ($2048 \to 38$), and softmax normalization. The highlighted `layer4` bottleneck serves as the Grad-CAM explainability target layer.

1. **Adaptive Average Pooling:** An `AdaptiveAvgPool2d((1, 1))` collapses spatial dimensions, producing a compact 2048-dimensional feature embedding vector independent of input resolution.
2. **Dropout Regularization:** A dropout layer with probability $p = 0.30$ randomly zeros activations during training, preventing co-adaptation of deep representations and improving generalization to out-of-distribution field samples.
3. **Linear Classification Layer:** A fully connected projection layer $\mathbf{W} \in \mathbb{R}^{2048 \times 38}$ with bias vector $\mathbf{b} \in \mathbb{R}^{38}$ maps deep latent features to unnormalized logit scores across all 38 plant health and disease classes:

$$\mathbf{z} = \mathbf{W}\mathbf{h} + \mathbf{b}, \quad \mathbf{z} \in \mathbb{R}^{38} \tag{4}$$

   where $\mathbf{h} \in \mathbb{R}^{2048}$ is the post-dropout embedding.

8.2.3 Parameter Counts and Resource Footprint

Table 4 provides the exact parameter breakdown for our primary ResNet-50 architecture and the EfficientNet-B0 comparison baseline.

Table 4: **Model Parameter and Layer Breakdown.**

Architecture	Backbone Params	Head Params	Total Params	Checkpoint Size
ResNet-50 (Ours)	23,508,032	77,862	**23,585,894**	~90.5 MB (`state_dict`)
EfficientNet-B0	4,007,548	48,678	**4,056,226**	~16.2 MB (`state_dict`)

8.3 Loss Function, Class Balancing & Optimizer Strategy

8.3.1 Weighted Cross-Entropy Loss with Label Smoothing

While our Phase 2 curation capped dominant classes at 1,000 images, minor agricultural diseases naturally have limited sample support in the training split (e.g., 152 samples in minority classes vs. 700 in capped classes). To ensure the model remains equally sensitive to rare, high-consequence crop infections, we implement class-weighted cross-entropy:

$$\mathcal{L}_{\text{CE}}(\mathbf{z}, y) = -w_y \sum_{c=1}^{C} q_c \log \left(\frac{\exp(z_c)}{\sum_{j=1}^{C} \exp(z_j)} \right) \tag{5}$$

where w_y is the inverse frequency class weight for true class y computed strictly on the training partition ($w_c \in$ `splits/class_weights.json`), and q_c represents label-smoothed targets:

$$q_c = (1 - \alpha) \cdot \mathbb{I}(c = y) + \frac{\alpha}{C} \tag{6}$$

with smoothing parameter $\alpha \in [0.0, 0.10]$. Label smoothing prevents the model from generating overconfident, saturated logit predictions, significantly improving post-softmax calibration on uncertain field photographs.

8.3.2 AdamW Optimizer & Decoupled Weight Decay

We select **AdamW** as our primary optimizer over standard SGD with momentum. AdamW decouples weight decay from the moving average gradient updates:

$$\theta_{t+1} = \theta_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta_t \lambda \theta_t \tag{7}$$

where $\lambda = 10^{-4}$ is the weight decay rate, η_t is the instantaneous learning rate, and $\hat{m}_t, \hat{v}_t$ are the bias-corrected first and second moment estimators. Decoupling ensures true L_2 regularization without distorting the adaptive gradient scaling.

8.3.3 Two-Stage Progressive Fine-Tuning with Cosine Annealing

Directly fine-tuning all layers simultaneously with a randomly initialized classification head can cause large, erratic gradient updates that destroy pretrained ImageNet feature detectors (catastrophic forgetting). We address this with a **Two-Stage Progressive Training Strategy**:

- **Stage 1: Head Warm-up (Epoch 1):** All backbone convolutional parameters are frozen via `set_backbone_trainable(model, False)` such that $\nabla_{\theta_{\text{backbone}}} \mathcal{L} = 0$. Only the randomly initialized weights of the custom head are optimized for one epoch with a linear learning rate warmup factor from $0.1 \times \eta_{\max}$ to $\eta_{\max} = 3 \times 10^{-4}$.
- **Stage 2: Full Fine-Tuning (Epochs 2–15):** The backbone is unfrozen (`set_backbone_trainable(model, True)`). All 23.5 M parameters are jointly fine-tuned using a **Cosine Annealing Schedule**:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{t - t_{\text{warmup}}}{T_{\max} - t_{\text{warmup}}} \pi \right) \right) \tag{8}$$

where $\eta_{\min} = 10^{-6}$ and $T_{\max} = 15$. This guarantees smooth learning rate decay toward zero, allowing fine convergence without manual step-decay tuning.

8.4 Systematic Hyperparameter Tuning Protocol (13 Trials)

To justify every architectural and optimization choice on empirical evidence rather than heuristic defaults, we implemented a structured 13-trial grid divided into two distinct search phases:

- **Stage A (Architectural Backbone & Learning Rate, Trials 1–6):** Explores the Cartesian product of backbones $\{\text{ResNet-50}, \text{EfficientNet-B0}\}$ across three learning rate orders of magnitude $\{10^{-3}, 3 \times 10^{-4}, 10^{-4}\}$, holding batch size at 64 and dropout at 0.3.
- **Stage B (Regularization & Capacity Ablation, Trials 7–13):** Takes the best-performing configuration from Stage A and benchmarks variations in dropout rate $(0.0, 0.5)$, batch size $(32, 128)$, higher weight decay (10^{-2}), label smoothing (0.10), and unweighted loss.

8.4.1 Efficiency Protocol

To maximize GPU resource efficiency on free cloud accelerators, each tuning trial is trained on a **20% stratified train subset** (`subset_frac=0.20`, 4,437 images) for **4 epochs**, and evaluated on the full, uncompromised validation split (4,747 images) using Macro-F1. Table 5 displays the structured trials and records the completed Stage A results.

Table 5: **Systematic 13-Trial Hyperparameter Tuning Grid and Results.**

ID	Stage	Backbone	LR	Batch	Drop	WD	Smooth	Weights	Val Acc	Val Macro-F1
1	A	ResNet-50	1×10^{-3}	64	0.3	1×10^{-4}	0.0	Yes	96.08%	0.9605
2	A	ResNet-50	3×10^{-4}	64	0.3	1×10^{-4}	0.0	Yes	**96.12%**	**0.9606**
3	A	ResNet-50	1×10^{-4}	64	0.3	1×10^{-4}	0.0	Yes	–	–
4	A	EfficientNet-B0	1×10^{-3}	64	0.3	1×10^{-4}	0.0	Yes	–	–
5	A	EfficientNet-B0	3×10^{-4}	64	0.3	1×10^{-4}	0.0	Yes	–	–
6	A	EfficientNet-B0	1×10^{-4}	64	0.3	1×10^{-4}	0.0	Yes	–	–
7	B	ResNet-50	3×10^{-4}	64	0.0	1×10^{-4}	0.0	Yes	–	–
8	B	ResNet-50	3×10^{-4}	64	0.5	1×10^{-4}	0.0	Yes	–	–
9	B	ResNet-50	3×10^{-4}	32	0.3	1×10^{-4}	0.0	Yes	–	–
10	B	ResNet-50	3×10^{-4}	128	0.3	1×10^{-4}	0.0	Yes	–	–
11	B	ResNet-50	3×10^{-4}	64	0.3	1×10^{-2}	0.0	Yes	–	–
12	B	ResNet-50	3×10^{-4}	64	0.3	1×10^{-4}	0.1	Yes	–	–
13	B	ResNet-50	3×10^{-4}	64	0.3	1×10^{-4}	0.0	No	–	–

9 Training Results, Evaluation & Bias Check

This section details Phase 4 of Deliverable 2, presenting our validation protocol, baseline training convergence dynamics, metric justifications, held-out test evaluation framework, error analysis methodologies, and the systematic Bias Check.

9.1 Validation Strategy and Data Hygiene

Our evaluation protocol strictly enforces the **70/15/15 stratified split** established during Phase 2:

- **Stratified Class Proportions:** Every disease and healthy category maintains identical class frequency distributions across the training (22,179 images), validation (4,747 images), and test (4,768 images) splits.

- **Zero Overlap & Data Leakage Prevention:** Deduplication via perceptual hashing (pHash) and exact MD5 matching guarantees that duplicate or near-duplicate leaf photographs cannot appear across separate partitions.
- **Validation for Model Selection Only:** All hyperparameter tuning trials, learning rate adjustments, and early stopping decisions are computed exclusively on the validation split.
- **Held-Out Test Set Isolation:** The test split (`splits/test.csv`) and real-world field set (`splits/field_test.csv`) are held out and evaluated strictly **once** on the finalized model architecture at the conclusion of training. Evaluating on the test split during tuning is strictly prohibited to avoid hyperparameter overfitting.
- **Justification Against K-Fold Cross-Validation:** For a dataset of ∼32,000 high-resolution images, full 5-fold cross-validation would require $5 \times 15 = 75$ epochs of ResNet-50 training (> 4 hours of compute). Given cloud GPU timeout quotas, a single fixed stratified 70/15/15 split provides high statistical stability while conserving compute budget.

9.2 Baseline Training Dynamics & Convergence Analysis

The baseline architecture (ResNet-50 with AdamW, LR $= 3 \times 10^{-4}$, weight decay 10^{-4}, and class-weighted loss) was trained for 15 epochs on a cloud NVIDIA Tesla T4 GPU. Table 6 details the epoch-by-epoch training and validation progression from `results/baseline/history.csv`.

Table 6: **Baseline ResNet-50 Epoch Progression (15 Epochs on Full Curated Dataset).**

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	Val Macro-F1	Learning Rate	Time
1*	3.5048	15.91%	3.3935	38.47%	0.3490	3.00×10^{-4}	208.3 s
2	0.4445	88.56%	0.2276	93.28%	0.9236	2.96×10^{-4}	233.3 s
3	0.1370	95.52%	0.1232	96.10%	0.9624	2.85×10^{-4}	227.4 s
4	0.0916	96.86%	0.0588	98.00%	0.9794	2.67×10^{-4}	230.3 s
5	0.0699	97.80%	0.0810	97.75%	0.9770	2.44×10^{-4}	234.0 s
6	0.0575	98.14%	0.0336	98.97%	0.9883	2.15×10^{-4}	234.7 s
7	0.0402	98.70%	0.0558	98.44%	0.9820	1.84×10^{-4}	229.7 s
8	0.0341	98.90%	0.0310	98.91%	0.9880	1.51×10^{-4}	232.6 s
9	0.0213	99.29%	0.0340	98.99%	0.9895	1.17×10^{-4}	233.0 s
10	0.0187	99.36%	0.0261	99.18%	0.9905	8.56×10^{-5}	230.5 s
11	0.0144	99.51%	0.0220	99.47%	0.9945	5.73×10^{-5}	230.7 s
12	0.0078	99.71%	0.0184	99.56%	0.9947	3.36×10^{-5}	229.3 s
13	0.0075	99.72%	0.0202	99.56%	0.9946	1.58×10^{-5}	230.2 s
14	0.0070	99.79%	**0.0169**	**99.64%**	**0.9961**	4.75×10^{-6}	225.0 s
15	**0.0057**	**99.81%**	0.0197	99.62%	0.9958	1.00×10^{-6}	230.6 s

*Epoch 1 represents the head-only warm-up phase with frozen backbone.

9.2.1 Key Training Observations

1. **Warmup Stabilization:** During Epoch 1, optimizing only the custom head brought validation accuracy from random chance ($1/38 \approx 2.63\%$) to 38.47% with zero disruption to the underlying ImageNet feature representations.
2. **Rapid Feature Alignment:** Unfreezing the backbone in Epoch 2 immediately produced an accuracy jump to 93.28% and a macro-F1 of 0.9236, demonstrating strong transferability

of pretrained residual filters to plant leaf textures.

3. **Generalization without Overfitting:** By Epoch 14, validation loss dropped to 0.0169 with validation accuracy reaching **99.64%** and macro-F1 achieving **0.9961**. The minimal gap between training accuracy (99.79%) and validation accuracy (99.64%) confirms that dropout ($p = 0.3$) and weight decay (10^{-4}) successfully controlled overfitting.

9.3 Evaluation Metrics & Agricultural Justifications

Evaluating agricultural disease models requires multi-dimensional performance assessment. Relying exclusively on raw accuracy can be dangerously misleading in agricultural decision support. We justify our core metrics below:

1. **Macro-Averaged Precision, Recall, and F1-Score:** Macro metrics assign equal weight to every disease class regardless of sample support:

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^{C} \frac{2 \cdot P_c \cdot R_c}{P_c + R_c} \tag{9}$$

   In contrast to weighted or micro averages that mask errors on rare classes, macro-F1 guarantees that failure on an infrequent but devastating disease (such as *Potato Early Blight* or *Apple Cedar Rust*) directly degrades the overall benchmark.

2. **High-Recall Priority for Pathogen Detection:** In agricultural field diagnosis, the asymmetric cost of misclassification is severe:

   - **False Negative (Low Recall):** The model misdiagnoses a diseased leaf as healthy or a benign condition. The farmer takes no action, leading to rapid pathogen spread, crop destruction, and severe financial loss.
   - **False Positive (Low Precision):** The model diagnoses disease on a healthy leaf. The farmer conducts an unnecessary spray application, incurring chemical costs but avoiding crop failure.

   Recall is therefore prioritized to eliminate missed infections.

3. **Macro One-vs-Rest AUC-ROC:** Evaluates class separability across all continuous prediction thresholds using softmax probabilities, ensuring confidence scores are reliably ranked.

4. **Top-3 Accuracy:** Field leaf symptoms often display overlapping visual phenotypes (e.g., fungal leaf mold vs. early blight). Top-3 accuracy measures whether the true disease is captured in the system's differential diagnosis list presented to the farmer in Phase 5.

9.4 Error Analysis Methodology

Upon final model selection on Day 5, Babar will execute `src/evaluate.py` to isolate failure modes:

1. **Confusion Matrix Inspection:** Identifying the top-10 most confused class pairs (e.g., *Corn Northern Leaf Blight* vs. *Gray Leaf Spot*).

2. **Worst-Performing Classes:** Tabulating the 5 classes with the lowest validation F1 scores and determining their primary confusion partners.

3. **Qualitative Failure Grid:** Extracting a visual grid of ≥ 12 misclassified images alongside model confidence scores to identify root visual causes (e.g., shadow occlusion, dual infection symptoms, or atypical leaf angles).

9.5 Bias Check Section

To ensure ethical and equitable AI deployment for farmers, our evaluation includes a dedicated **Bias Check**:

1. **Class Imbalance Bias:** Calculating the Spearman rank correlation between training class size and final validation F1 score to verify that minority classes are not systematically neglected.
2. **Crop-Level Disparity:** Measuring performance grouped by crop genus (Tomato, Potato, Corn, Apple, Grape) to detect whether specific crops suffer from underrepresented feature diversity.
3. **Healthy vs. Diseased Bias:** Auditing the false-healthy rate (the proportion of diseased leaves incorrectly classified as healthy).
4. **Domain Shift (Lab vs. Field Gap):** Measuring the generalization gap between clean lab benchmark images and real-world outdoor field images from `splits/field_test.csv`.

10 Team Contributions & Task Distribution

In accordance with course deliverable requirements, Table 7 provides a transparent breakdown of individual contributions, module ownership, and verified development tasks across Deliverables 1 and 2.

10.1 Individual Development Hygiene

All team members maintain individual module ownership, develop on separate feature branches, submit peer-reviewed pull requests, and write automated test suites for their respective codebases. No single member implements another member's assigned modules, ensuring rigorous adherence to university academic integrity standards.

References

[1] Hughes, D. P., & Salathé, M. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics. *arXiv preprint arXiv:1511.08060*.

[2] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 770–778.

[3] Selvaraju, R. R., Cogswell, M., Das, A., Vedaldi, A., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *IEEE ICCV*, 618–626.

[4] Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR)*.

[5] Loshchilov, I., & Hutter, F. (2017). SGDR: Stochastic gradient descent with warm restarts. *International Conference on Learning Representations (ICLR)*.

[6] Singh, D., Jain, N., Jain, P., Kayal, P., Kumawat, S., & Batra, N. (2020). PlantDoc: A dataset for visual plant disease detection. In *Proceedings of the 7th ACM IKDD CoDS and 25th COMAD*, 249–253.

Table 7: **Verified Team Contribution and Module Ownership Table.**

Team Member	GitHub Handle	Owned Modules & Technical Deliverables	Phase
Abdul Ahad (Leader, 24L-3029)	`@ErenYeagerJinkD`	**Deliverable 2:** Model architecture definitions (`src/models/classifier.py`), custom classification head, class-weighted loss and label smoothing (`src/losses.py`), AdamW optimizer and cosine warmup scheduler (`src/optim.py`), model test suite (`tests/test_model.py`), YAML experiment configs (`configs/*.yaml`), baseline training execution (ResNet-50, 99.64% val acc), model weights hosting and download automation (`scripts/download_weights.py`), architecture diagram, full report drafting. **Deliverable 1:** Repository architecture, dependencies (`requirements.txt`), .gitignore hygiene, raw PlantVillage acquisition.	1–4
M. Anas (24L-3004)	`@manas243`	**Deliverable 2:** Training engine loop with mixed precision and gradient clipping (`src/engine.py`), CLI training orchestrator with checkpointing (`src/train.py`), hyperparameter tuning harness (`src/tune.py`), engine unit tests (`tests/test_engine.py`), training curves plotting (`src/plotting.py`). **Deliverable 1:** Dataset class and DataLoader factory (`src/data/dataset.py`), image transforms and augmentation pipeline (`src/data/transforms.py`), data quality and deduplication scan (`src/data/quality.py`).	1–4
M. Babar Shahzad (24L-3047)	`@mbabar244`	**Deliverable 2:** Evaluation metrics suite (`src/metrics.py`), CLI evaluation module (`src/evaluate.py`), metric tests (`tests/test_metrics.py`), error analysis and confusion matrix extraction (`src/analysis/error_analysis.py`), systematic Bias Check auditing (`src/analysis/bias_check.py`). **Deliverable 1:** Field robustness image collection ($\geq$ 150 images), field metadata mapping (`field_meta/`), treatment knowledge base curation and JSON schema validation (`knowledge_base/`).	1–4